

提示词工程：从基础设计到高级推理技术

提示词工程（Prompt Engineering）是指通过设计、组织和优化输入内容，引导大语言模型生成更准确、更稳定、更符合业务要求的结果。

很多人会将提示词简单理解为“向模型提问”。但在真实的大模型应用中，提示词不仅包含问题，还可能包含角色定义、任务目标、背景知识、输出格式、约束条件、示例数据、工具说明和质量标准。一个成熟的提示词，本质上是一份面向大语言模型的任务说明书。

提示词工程的目标并不是让提示词看起来更复杂，而是降低任务歧义，让模型清楚地知道需要完成什么、依据什么完成、不能做什么，以及最终应该以什么形式输出。

提示词工程的基本原理

大语言模型根据当前上下文预测后续最可能出现的内容。模型并不会像传统程序一样严格执行代码，而是根据提示词中的语言模式、任务描述和上下文信息生成结果。

因此，提示词越模糊，模型需要自行补充的信息就越多，输出的不确定性也越高。例如，下面的提示词没有说明目标用户、内容范围和输出格式：

Plain Text

1 介绍一下 RAG。

模型可能生成一段简短定义，也可能生成一篇技术文章，还可能重点介绍向量数据库。由于任务边界不明确，不同调用之间的输出差异可能很大。

更加完整的提示词可以写成：

Plain Text

- 1 你是一名大模型应用开发讲师。
- 2
- 3 请面向具有 Python 基础、但不了解大模型应用开发的学习者，
- 4 介绍检索增强生成 RAG 的基本原理。
- 5
- 6 内容需要包括：
- 7 - RAG 解决的问题
- 8 - 基本处理流程
- 9 - 文档切分、向量检索和重排序的作用
- 10 - RAG 与模型微调的区别
- 11 - 一个简单的企业知识库案例
- 12
- 13 使用中文输出。
- 14 正文控制在 1500 字以内。
- 15 只使用一个一级标题，其余使用二级标题和三级标题。

这个版本明确了模型角色、目标读者、任务范围、内容结构、语言和篇幅，因此输出通常更加稳定。

技术	适用场景	主要价值	主要成本
Zero-shot	简单分类、翻译、摘要	提示短、调用简单	复杂边界不稳定
Few-shot	固定格式、业务分类	学习任务模式	占用上下文
推理提示	数学、逻辑、复杂判断	提高可验证性	延迟和输出长度增加
Meta Prompting	优化提示词、设计任务模板	自动化设计	仍需真实测试
Self-Consistency	高价值复杂推理	提高稳定性	多次调用成本较高
Knowledge Prompting	先建立分析框架	提升分析完整性	生成知识可能有误
Prompt Chaining	长任务、复杂流程	可控、可调试	系统复杂度增加
Tree of Thoughts	规划、方案设计	探索多个路径	分支成本高
RAG	企业知识、实时资料问答	提供外部依据	依赖检索质量
Tool-use	搜索、计算、数据库操作	扩展模型能力	需要权限和异常控制
PAL	精确计算、数据处理	降低计算错误	需要安全执行环境
ReAct	多步骤 Agent 任务	动态决定下一步	容易循环或滥用工具
Reflexion	错误修复、持续改进	利用反馈调整策略	依赖有效反馈
Multimodal CoT	图表、图片、截图分析	综合视觉与文本信息	受图片质量影响
Graph Prompting	关系网络、依赖分析	清晰表示复杂关系	需要结构化图数据

一个常见的选择原则是：先使用最简单的方法建立基线，再根据错误类型增加复杂机制。如果零样本提示已经能够达到要求，就不需要引入思维树或多轮 Agent。如果主要错误来自业务分类边界，可以增加少样本示例。如果主要错误来自缺少知识，可以使用 RAG。如果主要错误来自计算不准确，可以使用 PAL 或计算工具。如果任务包含多个相互依赖的步骤，可以使用提示链或 ReAct。

一个完整提示词的组成

实际项目中的提示词通常由多个部分组成。并不是每个提示词都必须包含所有部分，但任务越复杂，越需要明确描述。

提示词组成	作用	示例
角色定义	指定模型采用的专业视角	你是一名资深数据分析师
任务目标	明确模型需要完成什么	分析问卷结果中的群体差异
背景信息	提供完成任务所需的上下文	问卷包含 140 份真人回答和 100 份虚拟人回答
输入数据	给出需要处理的内容	问卷选项比例、样本数量和分组信息
执行步骤	规定分析或生成过程	先检查数据质量，再计算差异指标
约束条件	限制模型不能做的事情	不得编造缺失数据
输出格式	规定最终结果结构	使用 JSON 或 Markdown 表格输出
质量标准	说明什么样的结果算合格	每个结论必须包含数据依据
示例	展示期望的输入输出模式	提供一个标准结果样例

在简单任务中，角色定义并不总是必要。例如，将一句中文翻译成英文时，直接说明翻译要求即可。但在专业分析、内容生成、代码审查和复杂推理任务中，明确角色通常有助于模型选择合适的表达方式和知识范围。

编写提示词的基本原则

明确描述任务目标

提示词首先要回答一个问题：模型最终需要交付什么。

“分析这份数据”并不是一个清晰的任务，因为分析可能包括描述性统计、异常值检测、相关性分析、聚类分析或预测建模。

更明确的表达方式是：

Plain Text

1

分析不同年龄段用户对五个产品功能的选择比例，

2

识别差异最大的功能，并判断差异是否具有统计意义。

任务目标应尽量使用具体动作，例如提取、分类、比较、判断、总结、改写、生成、验证或排序。

提供足够的上下文

模型缺少背景信息时，会根据常见情况自行推测。适当提供上下文可以减少错误假设。

例如，在分析用户反馈时，可以说明产品类型、目标用户、反馈来源和分析目的：

Plain Text

1

这些反馈来自一款面向大学生的学习计划软件。

2

分析目的是确定下一版本需要优先优化的三个功能。

上下文并不是越多越好。与任务无关的信息会占用上下文空间，还可能干扰模型判断。应优先保留能够影响输出结论的信息。

使用清晰的边界标记

当提示词中包含长文本、示例、代码或多个数据块时，应使用明显的分隔符。

Plain Text

1

请分析下面的用户反馈。

2

3

<feedback>

4

{{ user_feedback }}

5

</feedback>

6

7

输出用户的主要问题、情绪倾向和改进建议。

常见的边界标记包括 XML 标签、Markdown 代码块、三引号以及自定义标签。标签名称最好能够说明内容含义，例如 `<survey_data>`、`<document>` 和 `<requirements>`。

明确输出格式

输出格式越明确，模型结果越容易被程序解析。

Plain Text

```
1 请按照以下JSON格式输出：
2
3 {
4   "category": "问题类别",
5   "sentiment": "positive | neutral | negative",
6   "summary": "问题摘要",
7   "priority": 1
8 }
```

在要求 JSON 输出时，还应说明字段类型、枚举值和空值规则。

字段	类型	可选值或规则
category	string	从预设问题类别中选择
sentiment	string	positive、neutral、negative
summary	string	不超过 100 字
priority	integer	1 至 5
evidence	array[string]	至少包含一条原文依据

仅给出 JSON 示例仍然可能存在歧义。例如，模型可能增加额外字段，或者使用 Markdown 代码块包装 JSON。因此可以补充说明：

Plain Text

```
1 只输出合法JSON，不输出解释文字，不使用Markdown代码块，不增加未定义字段。
```

将重要约束写成可执行规则

“结果要真实”“内容要专业”“分析要深入”等要求过于抽象。更有效的方式是将抽象标准转化为可检查规则。

例如，“分析要深入”可以改写为：

Plain Text

- 1 每个结论必须包含：
- 2 1. 观察到的现象；
- 3 2. 对应的数据依据；
- 4 3. 可能的原因；
- 5 4. 不能直接得出的结论。

“不要编造”可以改写为：

Plain Text

- 1 当输入信息不足时，将对应字段设置为null，并在limitations字段中说明缺失信息，不得根据常识补全具体数据。

Zero-shot Prompting 零样本提示

零样本提示是指不提供示例，直接描述任务，让模型依靠已有能力完成任务。

Plain Text

- 1 判断下面评论的情绪倾向，只能输出 positive、neutral 或 negative。
- 2
- 3 评论：这款软件功能挺多，但是启动速度实在太慢了。

可能输出：

Plain Text

- 1 negative

零样本提示适合任务规则简单、模型比较熟悉、输出类别明确的场景，例如情感分类、关键词提取、文本摘要、语言翻译和常见知识问答。

零样本提示的优点是提示词短、开发成本低、占用上下文少。缺点是当分类边界比较模糊，或者业务规则与通用理解不一致时，模型容易产生偏差。

例如，“退款咨询”与“退款投诉”在不同企业中可能有不同定义。这种情况下，仅依赖零样本提示通常不够稳定。

Few-shot Prompting 少样本提示

少样本提示是在提示词中提供少量输入输出示例，让模型学习任务模式。

Plain Text

- 1 请将用户反馈分类为：功能问题、性能问题、账户问题、付费问题。
- 2
- 3 示例一：
- 4 输入：登录后一直显示验证码错误。
- 5 输出：账户问题
- 6
- 7 示例二：
- 8 输入：打开报表需要等待一分钟。
- 9 输出：性能问题
- 10
- 11 示例三：
- 12 输入：为什么自动续费没有提前提醒？
- 13 输出：付费问题
- 14
- 15 现在分类：
- 16 输入：导出的Excel文件缺少最后两列。
- 17 输出：

少样本提示并不会真正更新模型参数，而是通过上下文示例让模型理解分类标准、表达风格和输出结构。

示例质量通常比示例数量更加重要。好的示例应该覆盖典型情况、容易混淆的边界情况和特殊情况。示例之间还应保持格式一致，避免一个示例输出类别名称，另一个示例输出完整解释。

示例应尽量真实，且与实际输入具有相似的长度和复杂度。如果实际任务输入是长篇客户投诉，而示例只有几个字，模型可能无法学习如何处理复杂信息。

对于分类任务，可以重点提供以下类型的示例：

示例类型	作用
典型示例	展示每个类别最常见的特征
边界示例	区分容易混淆的类别
否定示例	说明什么情况不属于某类别
多标签示例	说明一条数据包含多个问题时如何处理
缺失信息示例	说明信息不足时如何输出

示例的排列也可能影响结果。为了降低顺序偏差，可以让不同类别交替出现，避免某个类别始终位于最后。

Chain-of-Thought Prompting 推理提示

推理提示用于引导模型在回答复杂问题时进行分步骤分析。它适合数学计算、逻辑判断、多条件决策、代码调试和复杂业务分析。

在实际应用中，与其要求模型输出完整的内部思考过程，更推荐要求模型输出可核查的关键步骤、计算依据和最终结论。

Plain Text

1 请解决下面的问题。

2

3 要求：

4 - 列出使用的公式；

5 - 展示关键计算步骤；

6 - 检查单位是否一致；

7 - 最后单独输出结论。

这种方式能够提升结果的可验证性，同时避免生成大量冗长、不可控的推理文本。

例如：

Plain Text

- 1 一个商品原价200元，先打八折，再使用满150减20优惠券。
- 2 计算最终支付金额和总优惠比例。
- 3
- 4 请输出：
- 5 - 折扣后价格
- 6 - 优惠券后价格
- 7 - 总优惠金额
- 8 - 总优惠比例

对于简单分类或信息提取任务，没有必要强制模型进行复杂推理。过度使用推理提示可能增加延迟、成本和输出噪声。

Meta Prompting 元提示

元提示关注的不是具体答案，而是任务的结构、解决方法和提示词本身。它可以让模型帮助设计、分析或优化另一个提示词。

例如：

Plain Text

- 1 你是一名提示词优化专家。
- 2
- 3 请分析下面的提示词，并从以下方面进行改进：
- 4 - 任务目标是否明确；
- 5 - 输入信息是否完整；
- 6 - 输出格式是否可执行；
- 7 - 是否存在相互冲突的约束；
- 8 - 是否容易引发模型编造；
- 9 - 是否缺少异常情况处理。
- 10
- 11 输出改进后的完整提示词。
- 12
- 13 <original_prompt>
- 14 {{ original_prompt }}
- 15 </original_prompt>

元提示适合构建提示词优化工具、自动生成任务模板和检查提示词质量。

但模型生成的提示词仍然需要测试。提示词看起来更加完整，并不代表在真实数据上一定更加稳定。最终效果应通过测试集、错误案例和业务指标进行评估。

Self-Consistency 自一致性

自一致性是一种通过多次生成和结果聚合提高可靠性的方法。基本思想是让模型针对同一个问题生成多个候选结果，然后选择出现频率最高或综合评分最高的答案。

例如，在一个复杂数学问题中，可以让模型独立求解五次，再比较最终答案。如果多数结果一致，可以将其作为最终结果。

自一致性不仅适用于数学推理，还可以用于分类、信息抽取、方案评估和风险判断。

Plain Text

1 请使用三种相互独立的方法分析这个问题。

2 分别给出每种方法的结论和依据。

3 最后比较三个结果，输出一致结论与存在争议的部分。

在工程系统中，自一致性通常由外部程序实现。程序并行调用模型多次，再通过投票、规则判断或另一个模型进行结果聚合。

这种方法能够提高复杂任务的稳定性，但会增加调用成本和响应时间。因此更适合高价值、高风险任务，而不是所有请求。

Generate Knowledge Prompting 知识生成提示

知识生成提示是先让模型生成与问题相关的背景知识，再基于这些知识完成任务。

例如，直接要求模型分析某个用户为什么会流失，模型可能只给出泛泛原因。可以将任务拆成两个阶段：

Plain Text

1 请先总结影响订阅制软件用户流失的常见因素。

2 只输出可能影响用户决策的因素，不分析当前案例。

然后再输入具体案例：

Plain Text

```
1 根据前面总结的用户流失因素，分析下面用户的流失原因。
2
3 <user_profile>
4 {{ user_profile }}
5 </user_profile>
6
7 <usage_data>
8 {{ usage_data }}
9 </usage_data>
```

这种方法适用于模型需要先建立分析框架，再处理具体案例的场景。

需要注意，模型自己生成的知识仍然可能存在错误。对于医疗、法律、金融、产品规格和实时政策等高精度任务，应优先使用经过验证的外部知识，而不是完全依赖模型内部生成。

Prompt Chaining 提示链

提示链是将一个复杂任务拆分成多个相互连接的子任务，每一步的输出作为下一步的输入。

例如，生成一份行业研究报告可以拆分为：

Plain Text

- 1 第一步：从资料中提取关键事实。
- 2 第二步：将事实按照市场规模、竞争格局、用户需求和风险进行分类。
- 3 第三步：根据分类结果生成报告大纲。
- 4 第四步：按照大纲撰写正文。
- 5 第五步：检查正文中的事实是否都能在资料中找到依据。

与一次性要求模型完成所有任务相比，提示链具有更好的可控性。开发者可以检查每个中间结果，定位错误发生在哪一步，也可以针对不同步骤使用不同模型。

方式	优点	缺点
单次提示	调用简单、延迟较低	错误难定位，复杂任务容易遗漏
提示链	可检查、可调试、结构清晰	调用次数更多，流程更复杂
并行提示	处理速度快，可生成多个候选	需要额外的结果合并逻辑
条件提示链	可以根据中间结果选择不同路径	状态管理和异常处理较复杂

设计提示链时，需要明确每一步的输入输出协议。最好让中间结果采用结构化格式，避免下一步无法稳定解析。

Tree of Thoughts 思维树

思维树是一种探索多个候选推理路径的方法。模型不是沿着单一路径生成答案，而是先提出多个可能方案，再对方案进行评估、扩展和筛选。

例如，在制定产品增长方案时，可以先生成多个方向：

Plain Text

1 针对当前用户增长问题，提出五种差异明显的解决方向。

2

3 每种方向包含：

4 - 核心策略

5 - 适用前提

6 - 预期收益

7 - 执行成本

8 - 主要风险

然后再评价这些方向：

Plain Text

1 根据预算、实施周期、风险和预期收益，对五种方案进行评分。

2 淘汰明显不可行的方案，并对得分最高的两个方案继续细化。

思维树适合开放式规划、复杂决策、代码方案设计和创意任务。它的优势是能够避免模型过早锁定第一个答案。

但思维树并不保证一定得到正确答案。候选方案的质量、评价标准和筛选机制都会影响最终结果。在工程实现中，通常需要控制分支数量和搜索深度，避免成本快速增长。

Retrieval-Augmented Generation 检索增强生成

检索增强生成（Retrieval-Augmented Generation，RAG）通过从外部知识库检索相关内容，并将检索结果提供给模型，从而降低模型仅依赖内部知识产生错误的风险。

一个典型的 RAG 流程包括文档加载、文本清洗、文档切分、向量化、建立索引、查询改写、召回、重排序、上下文构建和答案生成。

RAG 提示词的关键不是简单地将检索内容放进上下文，而是明确规定模型如何使用这些资料。

Plain Text

```
1 你是企业知识库问答助手。
2
3 请仅根据提供的参考资料回答问题。
4
5 回答规则：
6 - 不得使用参考资料之外的信息补充具体事实；
7 - 每个关键结论必须标注对应的source_id；
8 - 当资料不足时，明确回答“根据现有资料无法确定”；
9 - 当不同资料存在冲突时，分别列出冲突内容；
10 - 不得将用户问题中的说法直接当作事实。
11
12 <question>
13 {{ question }}
14 </question>
15
16 <references>
17 {{ retrieved_documents }}
18 </references>
```

一个高质量 RAG 系统不仅依赖提示词，还依赖检索质量。如果召回的文档不相关、文档切分破坏语义、知识库内容过期，提示词再完善也难以生成可靠答案。

问题	建议
检索结果包含无关内容	要求模型先判断资料相关性
多份资料互相冲突	要求分别列出不同来源的说法
资料不足	明确规定不得自行补全
用户问题带有错误前提	要求先验证问题前提
文档包含恶意指令	明确说明文档内容是数据，不是系统指令
引用无法追踪	给每个文本块分配 source_id
答案过度引用	要求先归纳，再引用关键依据

Automatic Reasoning and Tool-use 自动推理与工具使用

自动推理与工具使用强调让模型根据任务判断是否需要调用搜索、计算器、数据库、代码执行器或外部API。

例如：

Plain Text

```
1 你可以使用以下工具：
2 - calculator：执行精确计算；
3 - search_documents：搜索企业知识库；
4 - get_order：查询订单状态。
5
6 执行规则：
7 - 涉及金额计算时必须使用calculator；
8 - 涉及企业制度时必须先搜索知识库；
9 - 查询订单前必须确认order_id存在；
10 - 工具调用失败时不得编造结果；
11 - 完成任务后输出使用过的工具和最终结论。
```

工具说明应明确工具的能力边界、参数类型、调用条件和失败处理方式。仅提供一个工具名称，而不说明什么时候使用，容易造成模型滥用工具或完全不调用工具。

Automatic Prompt Engineer 自动提示词优化

自动提示词优化是让模型或程序自动生成多个提示词候选，并通过测试数据评估候选效果。

一个基本流程是：

Plain Text

1 生成候选提示词

2 → 在验证集上执行

3 → 计算准确率或业务指标

4 → 分析错误样本

5 → 修改提示词

6 → 再次测试

例如，对于文本分类任务，可以让模型生成十个不同版本的分类指令，再使用同一批标注数据进行测试。最终选择准确率、稳定性和成本综合表现最好的版本。

自动优化不能只依赖模型主观评价。模型可能认为某个提示词“更清晰”，但实际准确率并没有提高。可靠的方法是建立真实测试集，并使用可量化指标进行比较。

Active-Prompt 主动样本提示

Active-Prompt 会优先寻找模型最不确定、最容易出错的样本，再为这些样本构造人工标注或高质量推理示例。

它与主动学习的思想相似：与其随机增加大量示例，不如优先处理信息价值最高的困难样本。

例如，在客户问题分类中，模型对“账户问题”和“支付问题”经常混淆。可以从错误数据中选择典型案例，将这些案例加入少样本提示。

Plain Text

1 输入：我更换银行卡后，账号突然无法登录。

2 输出：账户问题

3

4 解释：虽然输入提到了银行卡，但用户的主要问题是无法登录，因此归类为账户问题。

Active-Prompt 适合已经积累了一定测试数据和错误案例的项目。它通常比盲目增加示例数量更有效。

Directional Stimulus Prompting 方向性刺激提示

方向性刺激提示通过提供关键词、提示线索、主题标签或重点方向，引导模型关注特定信息。例如，在生成新闻摘要时，可以先给出需要重点关注的方向：

Plain Text

1 摘要重点：

2 - 政策变化

3 - 对中小企业的影响

4 - 生效时间

5 - 企业需要采取的行动

6

7 请根据以上重点摘要下面的政策文件。

这种方法适合摘要、内容生成、信息提取和观点分析。它不会直接给出完整答案，而是告诉模型“应该重点关注什么”。

方向性提示应避免暗示不存在的结论。例如，不能在没有证据的情况下要求模型重点分析“该政策造成的负面影响”，因为这可能引导模型强行寻找负面信息。更中立的表达是“分析可能产生的正面和负面影响”。

Program-Aided Language Models 程序辅助语言模型

程序辅助语言模型（Program-Aided Language Models，PAL）让模型负责理解问题和生成计算逻辑，再由程序执行精确计算。

例如：

Plain Text

1 请将下面的问题转换为Python计算代码。

2 只负责建立变量和计算公式，不要直接心算最终结果。

模型生成：

Python

```
1 original_price = 200
2 discounted_price = original_price * 0.8
3 final_price = discounted_price - 20
4 discount_rate = (original_price - final_price) / original_price
```

然后由 Python 执行代码得到精确结果。

PAL 适合数学计算、日期处理、统计分析、数据转换和复杂公式计算。模型擅长理解自然语言，但不一定擅长稳定完成多步精确计算。将语言理解与程序执行结合，可以减少算术错误。

使用 PAL 时，需要对生成代码进行安全限制。不能直接在不受控环境中执行任意代码，应限制可调用的库、文件权限、网络权限和执行时间。

ReAct：推理与行动结合

ReAct 结合 Reasoning 和 Acting，让模型在完成任务时交替进行分析、调用工具、观察结果和继续决策。

一个典型流程是：

Plain Text

- 1 分析当前任务
- 2 → 选择工具
- 3 → 调用工具
- 4 → 获取观察结果
- 5 → 判断是否需要继续
- 6 → 输出最终答案

例如，用户询问某个订单为何延迟，模型可能先查询订单状态，再查询物流信息，最后根据工具返回结果生成解释。

在实际系统中，可以将内部决策与用户可见输出分开。用户通常只需要看到简洁的执行结果、使用的依据和异常说明，不需要看到所有内部推理文本。

ReAct 提示词需要明确停止条件，否则模型可能重复调用工具。

Plain Text

- 1 停止规则：
- 2 - 已获得回答问题所需的全部信息后立即停止；
- 3 - 同一个工具不得使用完全相同的参数重复调用；
- 4 - 连续两次工具调用没有获得新信息时停止；
- 5 - 达到最大工具调用次数后输出当前结果和缺失信息。

Reflexion 反思机制

Reflexion 让模型根据任务结果、错误反馈或评价信息，总结失败原因，并在下一轮任务中调整策略。

例如，模型第一次生成 SQL 时出现字段错误，可以向模型提供数据库报错：

Plain Text

- 1 上一次SQL执行失败。
- 2
- 3 错误信息：
- 4 Unknown column 'user_type' in 'field list'
- 5
- 6 请完成以下任务：
- 7 - 判断错误原因；
- 8 - 根据数据库字段定义修改SQL；
- 9 - 不得使用不存在的字段；
- 10 - 输出修改后的SQL。

反思机制适合代码生成、Agent 任务、规划任务和多轮内容优化。

反思不能只写“请反思一下”。应提供明确的反馈信号，例如测试结果、评分、错误日志、人工意见或规则检查结果。没有外部反馈的自我反思，可能只是生成另一种看似合理的解释，并不一定真正提高准确性。

Multimodal CoT 多模态推理

多模态推理是指模型同时处理文本、图片、图表、截图、音频或视频等不同形式的信息。

例如，在分析统计图表时，可以要求模型按照固定步骤输出：

Plain Text

- 1 请分析图片中的统计图表。
- 2
- 3 分析要求：
- 4 - 识别图表类型；
- 5 - 提取横轴、纵轴和图例含义；
- 6 - 描述最高值、最低值和主要变化趋势；
- 7 - 区分图中直接可见的事实与推测；
- 8 - 无法清晰识别的数字不要猜测；
- 9 - 最后总结图表可以支持的结论。

多模态提示词应明确模型需要关注的视觉区域和信息类型。例如，可以说明“只分析右侧柱状图，不分析页面中的导航栏”。

对于模糊图片、被截断的图表和低分辨率截图，模型可能无法准确识别数字。提示词应允许模型输出“无法识别”，而不是强制给出具体结果。

Graph Prompting 图提示

图提示使用节点、关系和路径表示复杂信息。它适合知识图谱、社交关系、因果关系、依赖关系和流程网络等任务。

例如，可以将系统依赖关系表示为：

Plain Text

- 1 节点：
- 2 A：用户服务
- 3 B：订单服务
- 4 C：支付服务
- 5 D：库存服务
- 6
- 7 关系：
- 8 A -> B：创建订单
- 9 B -> C：发起支付
- 10 B -> D：扣减库存
- 11 C -> B：返回支付结果

然后要求模型分析：

Plain Text

- 1 根据节点和关系分析：
- 2 - 支付服务故障会影响哪些流程；
- 3 - 哪些服务可能成为单点故障；
- 4 - 是否存在循环依赖；
- 5 - 给出故障隔离建议。

图提示的优势是能够比长篇自然语言更清晰地表达实体之间的关系。对于复杂图结构，可以结合图数据库或图算法，先计算关键节点、最短路径和社区结构，再让模型解释结果。

提示词优化流程

提示词优化不应该只依赖个人感觉，而应当像软件开发一样建立测试、评估和迭代流程。

建立测试集

测试集需要覆盖正常样本、边界样本、异常样本和对抗样本。

例如，一个信息抽取系统的测试数据可以包括：

样本类型	示例
正常样本	信息完整、字段表达清晰
缺失样本	缺少日期或金额
多值样本	一段文本包含多个订单
冲突样本	前后出现不同金额
模糊样本	使用“下周”“月底”等相对时间
噪声样本	包含广告、签名和无关文本
对抗样本	文本中包含“忽略之前指令”

测试集不应全部由模型生成。最好加入真实业务数据和历史错误案例，因为真实数据通常包含人工难以提前想到的表达方式。

定义评价指标

不同任务需要不同指标。

分类任务可以使用准确率、精确率、召回率、F1 值和混淆矩阵。信息抽取可以使用字段准确率、字段级 F1 和格式合法率。生成任务可以评价事实准确性、完整性、相关性、语言质量和格式遵循率。

RAG 系统还需要分别评价检索和生成：

环节	可使用的指标
文档召回	Recall@K、Hit Rate
文档排序	MRR、NDCG
上下文质量	Context Precision、Context Recall
答案质量	Correctness、Relevance
事实依据	Faithfulness、Citation Accuracy
系统表现	延迟、Token 成本、失败率

仅评价最终答案，可能无法判断问题来自检索还是生成。因此复杂系统应分别记录每个阶段的结果。

分析错误类型

优化提示词之前，应先判断模型为什么出错。

常见错误可以分为：

错误类型	可能的改进方式
不理解任务	重写任务目标，减少歧义
输出格式错误	增加 Schema、示例和格式校验
分类边界混淆	增加边界示例和判断规则
遗漏信息	增加检查清单或分步骤处理
编造事实	使用 RAG，限制知识来源
计算错误	使用代码或计算工具
工具调用失败	增加参数校验和失败处理
长文本信息丢失	优化切分、摘要或上下文组织
指令冲突	调整指令优先级，删除重复规则
回答过于冗长	明确字数和输出结构

不要在每次出错后简单地增加一句限制。不断堆叠规则可能使提示词越来越长，并产生新的冲突。应优先修改最根本的任务定义和输出协议。